Tipos Abstratos de Dados Vetor, Pilha e Fila

Estruturas de Dados

adalberto@computacao.ufcg.edu.br

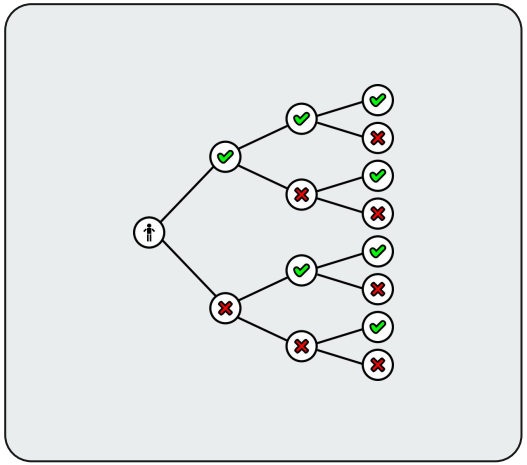
Tipos Abstratos de Dados

Permite especificação de um contrato

Definir um tipo formalmente

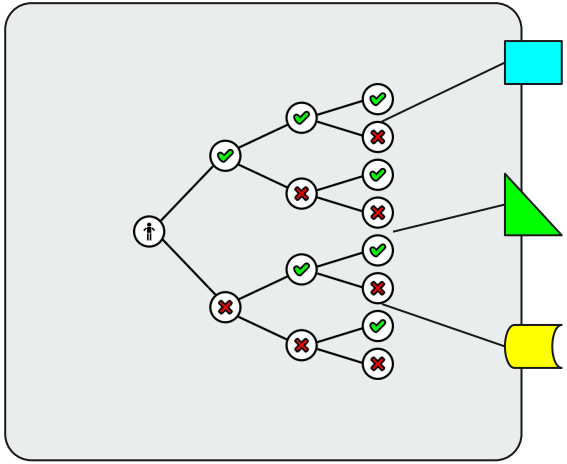
Tipos Abstratos de Dados

Encapsula alguma estrutura de dados



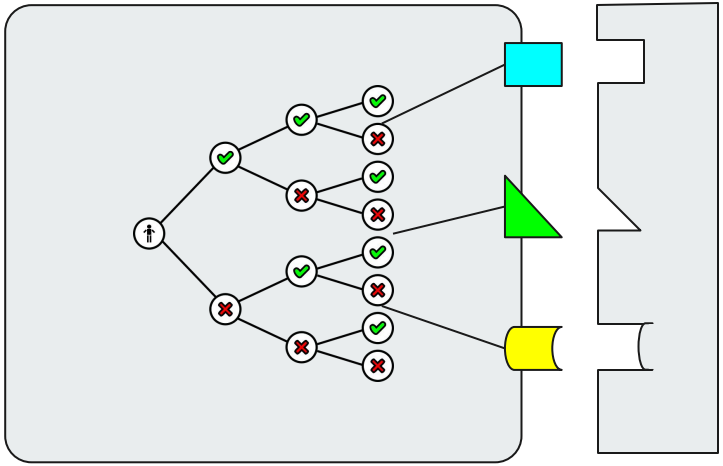
Tipos Abstratos de Dados

Define operações



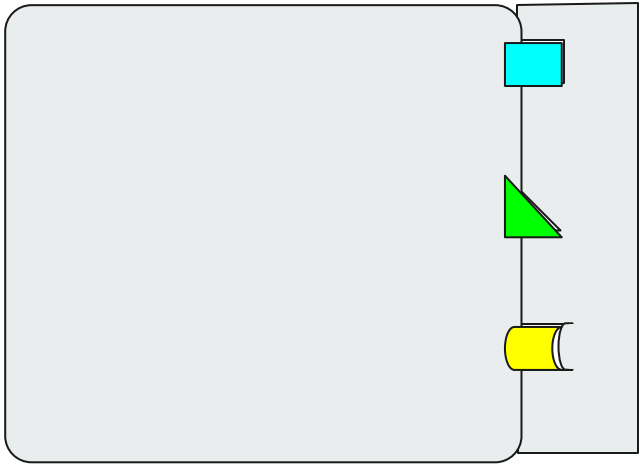
Tipos Abstratos de Dados

Define operações



Tipos Abstratos de Dados

Abstrai/esconde implementação específica



Tipos Abstratos de Dados

Como definir e implementar Tipos Abstratos de Dados em Java?

Tipos Abstratos de Dados

Como definir e implementar Tipos Abstratos de Dados em Java?

Programe para interfaces!!!

<<Interface>>

↑

Classe

Tipos Abstratos de Dados

Operações básicas:

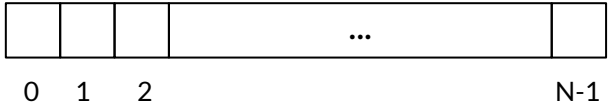
- inserir
- remover
- pesquisar
- contem
- tamanho
- estaVazio
- estaCheio

Vetor

Vetor

Operações básicas:

- inserir
- remover
- pesquisar
- pesquisar pela posição
- tamanho
- estaVazio
- estaCheio



Vetor

Operações básicas:

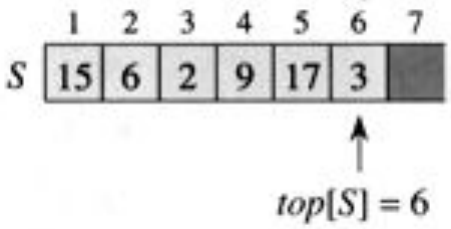
- inserir $\theta(1)$
- remover $\theta(1)$
- pesquisar $\theta(n)$
- pesquisar pela posição $\theta(1)$
- tamanho $\theta(1)$
- estaVazio $\theta(1)$
- estaCheio $\theta(1)$

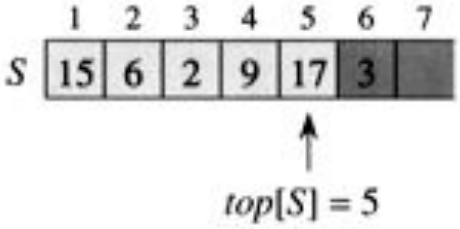
Pilha (Stack)

LIFO - Last-In First-Out

Pilha (Simulação)

Pilha





Pilha (Stack)

Operações básicas:

- inserir (**push**)
- remover (**pop**)
- **top** (peek)
- ~~pesquisar~~
- ~~contem~~
- estaVazio (isEmpty)
- estaCheio (isFull)

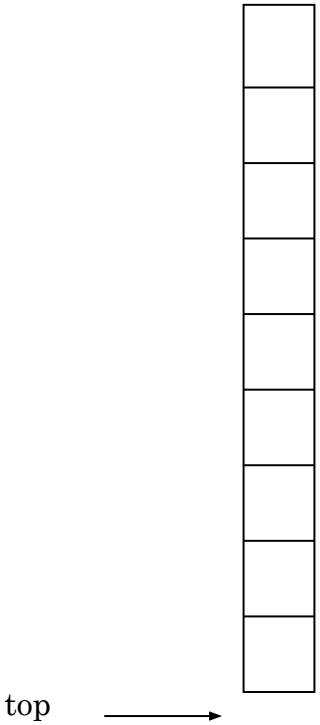
Pilha (Stack)

Aplicações:

- Programas recursivos
- Histórico de navegações
- Control + Z

Pilha (Stack)

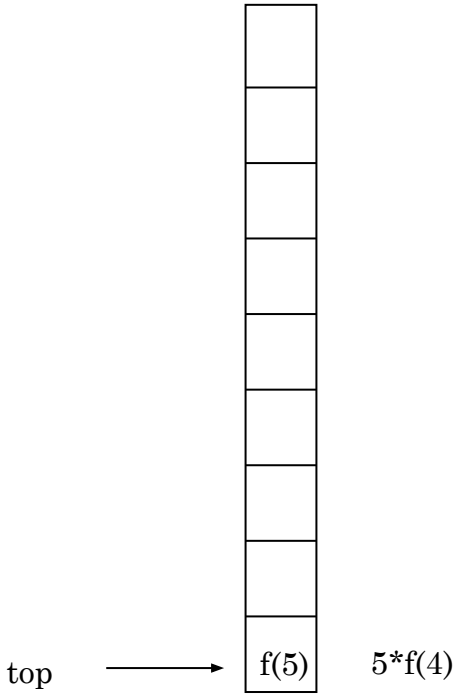
$f(0) = 1$
$f(n) = n*f(n-1)$



Pilha (Stack)

f(0) = 1
f(n) = n*f(n-1)

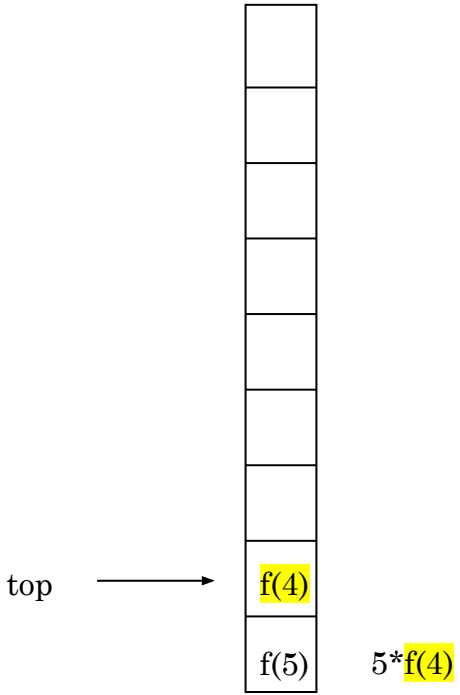
f(5) = 5*f(4)



Pilha (Stack)

f(0) = 1
f(n) = n*f(n-1)

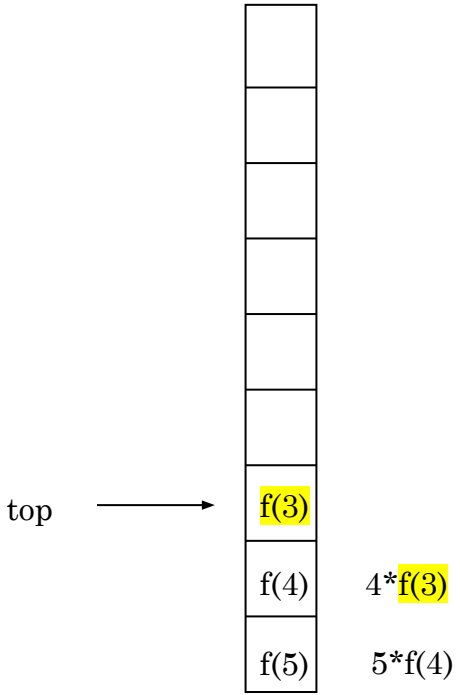
f(5) = 5*f(4)



Pilha (Stack)

f(0) = 1
f(n) = n*f(n-1)

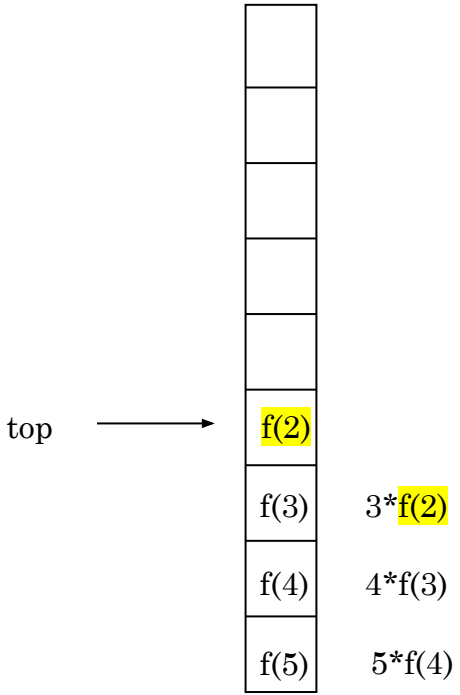
f(5) = 5*f(4)
f(4) = 4*f(3)



Pilha (Stack)

f(0) = 1
f(n) = n*f(n-1)

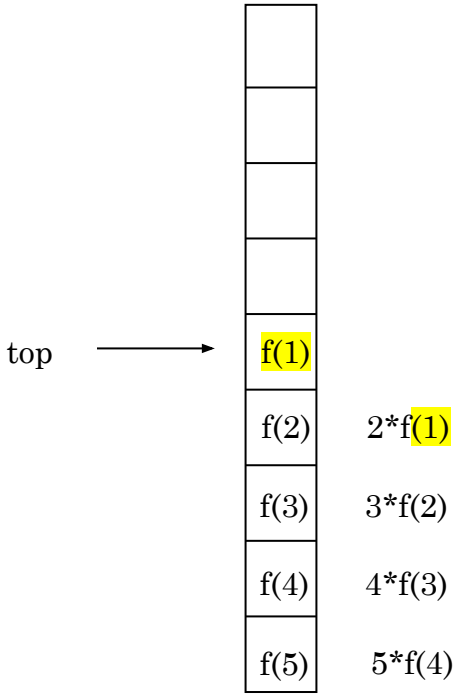
f(5) = 5*f(4)
f(4) = 4*f(3)
f(3) = 3*f(2)



Pilha (Stack)

f(0) = 1
f(n) = n*f(n-1)

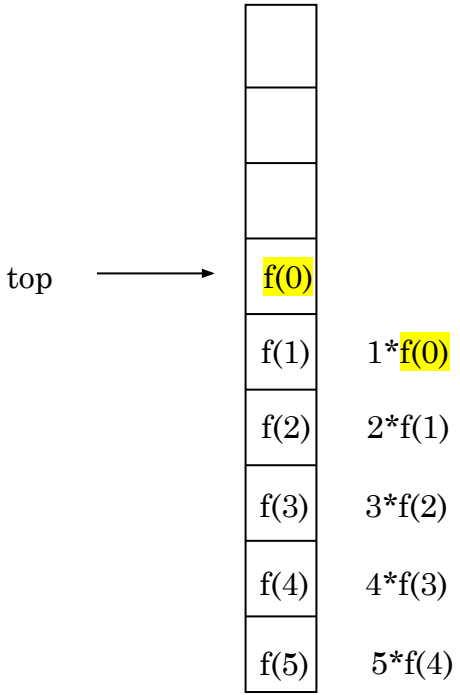
f(5) = 5*f(4)
f(4) = 4*f(3)
f(3) = 3*f(2)
f(2) = 2*f(1)



Pilha (Stack)

f(0) = 1
f(n) = n*f(n-1)

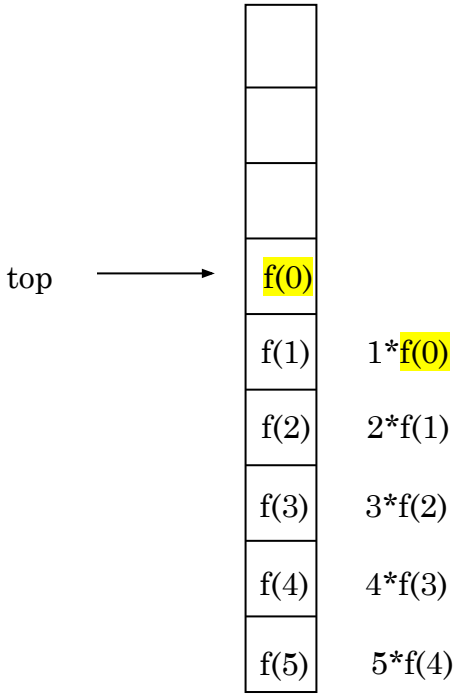
f(5) = 5*f(4)
f(4) = 4*f(3)
f(3) = 3*f(2)
f(2) = 2*f(1)
f(1) = 1*f(0)



Pilha (Stack)

f(0) = 1
f(n) = n*f(n-1)

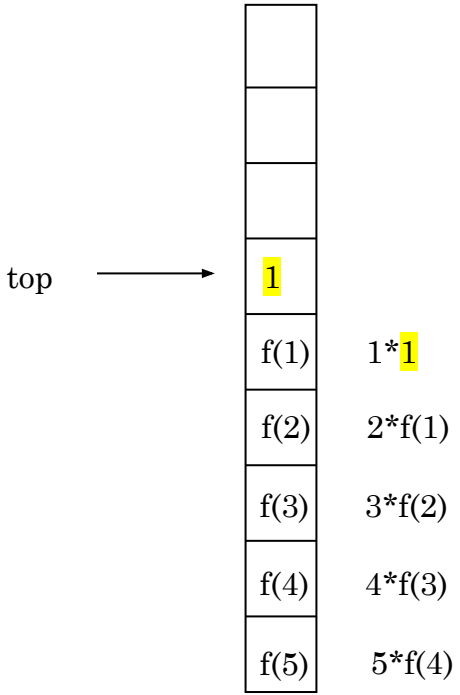
f(5) = 5*f(4)
f(4) = 4*f(3)
f(3) = 3*f(2)
f(2) = 2*f(1)
f(1) = 1*f(0)
f(0) = 1



Pilha (Stack)

f(0) = 1
f(n) = n*f(n-1)

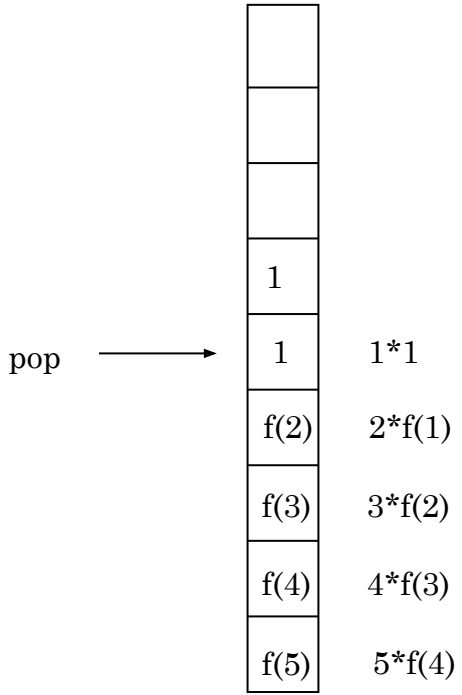
f(5) = 5*f(4)
f(4) = 4*f(3)
f(3) = 3*f(2)
f(2) = 2*f(1)
f(1) = 1*f(0)
f(0) = 1



Pilha (Stack)

f(0) = 1
f(n) = n*f(n-1)

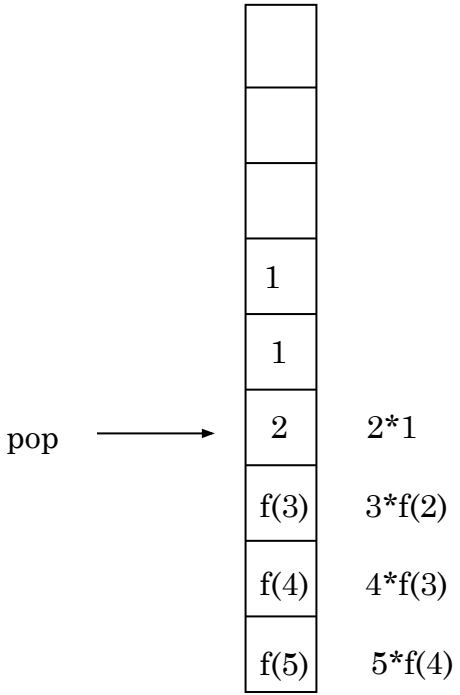
f(5) = 5*f(4)
f(4) = 4*f(3)
f(3) = 3*f(2)
f(2) = 2*f(1)
f(1) = 1*1



Pilha (Stack)

f(0) = 1
f(n) = n*f(n-1)

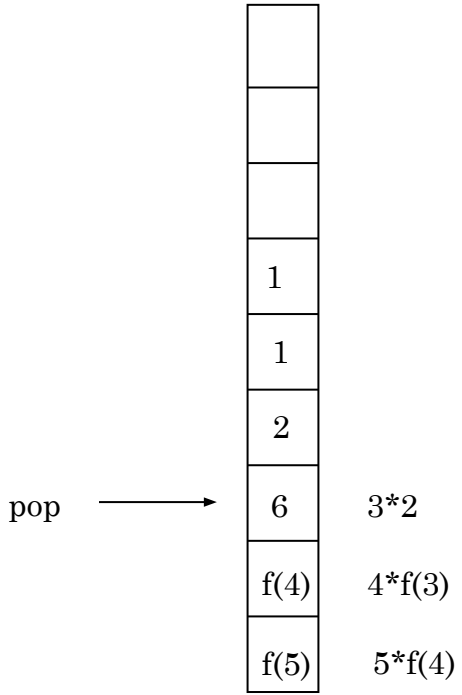
f(5) = 5*f(4)
f(4) = 4*f(3)
f(3) = 3*f(2)
f(2) = 2*1



Pilha (Stack)

f(0) = 1
f(n) = n*f(n-1)

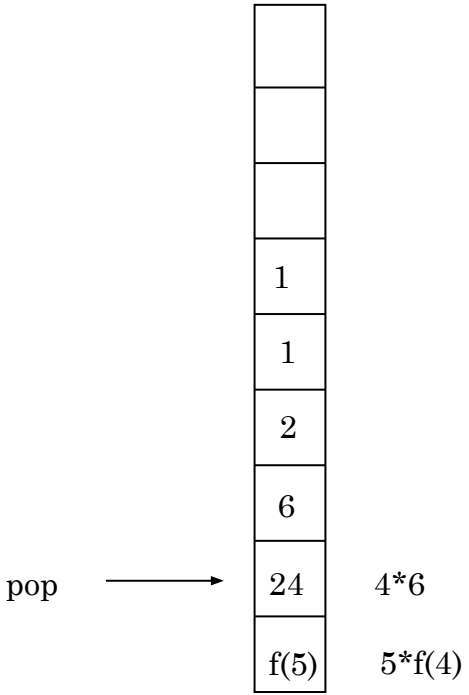
f(5) = 5*f(4)
f(4) = 4*f(3)
f(3) = 3*2



Pilha (Stack)

f(0) = 1
f(n) = n*f(n-1)

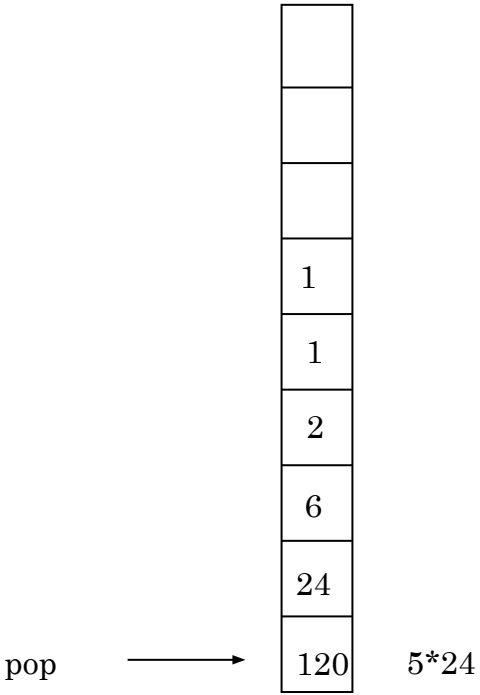
f(5) = 5*f(4)
f(4) = 4*6



Pilha (Stack)

f(0) = 1
f(n) = n*f(n-1)

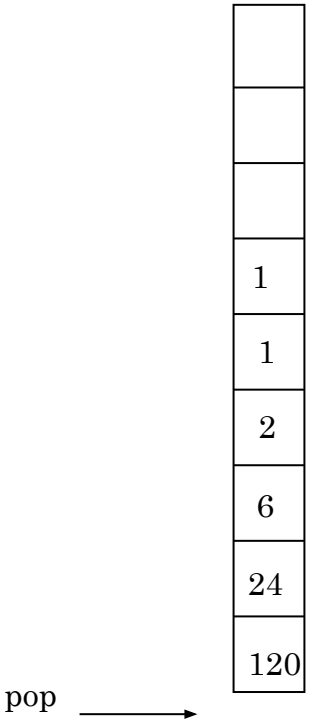
f(5) = 5*24



Pilha (Stack)

f(0) = 1
f(n) = n*f(n-1)

f(5) = 120



Pilha (Stack) - Implementação

```
public interface Stack<T> {
  public void push(T elem) throws StackOverflowException;
  public T pop() throws StackUnderflowException;
  public T top();
  public boolean isEmpty();
  public boolean isFull();
}
```

```
public class StackOverflowException
   extends Exception {
 public StackOverflowException() {
  super("Stack is full");
 }
}
```

```
public class StackUnderflowException
   extends Exception {
 public StackUnderflowException() {
  super("Stack is empty");
 }
}
```

Pilha (Stack) - Implementação

```
public class StackImpl<T> implements Stack<T> {
  private T[] array;
  private int top;

  public StackImpl(int size) {
    array = (T[]) new Object[size];
    top = -1;
  }
  public void push(T elem) throws StackOverflowException {...}
  public T pop() throws StackUnderflowException {...}
  public T top() {...}
  public boolean isEmpty() {...}
  public boolean isFull() {...}
}
```

Pilha (Stack) - Implementação

```
STACK-EMPTY(S)
if top[S] = 0
    then return TRUE
    else return FALSE
```

```
PUSH(S, x)
top[S] ← top[S] + 1
S[top[S]] ← x
```

```
POP(S)
if STACK-EMPTY(S)
    then error "underflow"
    else top[S] ← top[S] − 1
         return S[top[S] + 1]
```

Pilha (Stack) - Implementação

```
STACK-EMPTY(S)
if top[S] = 0
    then return TRUE
    else return FALSE
```

$\theta(1)$

```
PUSH(S, x)
top[S] ← top[S] + 1
S[top[S]] ← x
```

$\theta(1)$

```
POP(S)
if STACK-EMPTY(S)
    then error "underflow"
    else top[S] ← top[S] − 1
         return S[top[S] + 1]
```

$\theta(1)$

Pilha (Collections Framework)

Interface Deque<E>

All Known Implementing Classes:

ArrayDeque, ConcurrentLinkedDeque, LinkedBlockingDeque, LinkedList

Stack Method	Equivalent Deque Method
push(e)	addFirst(e)
pop()	removeFirst()
peek()	peekFirst()

Obs: Não use Stack!

Fila (Queue - kyü)

FIFO - First-In First-Out

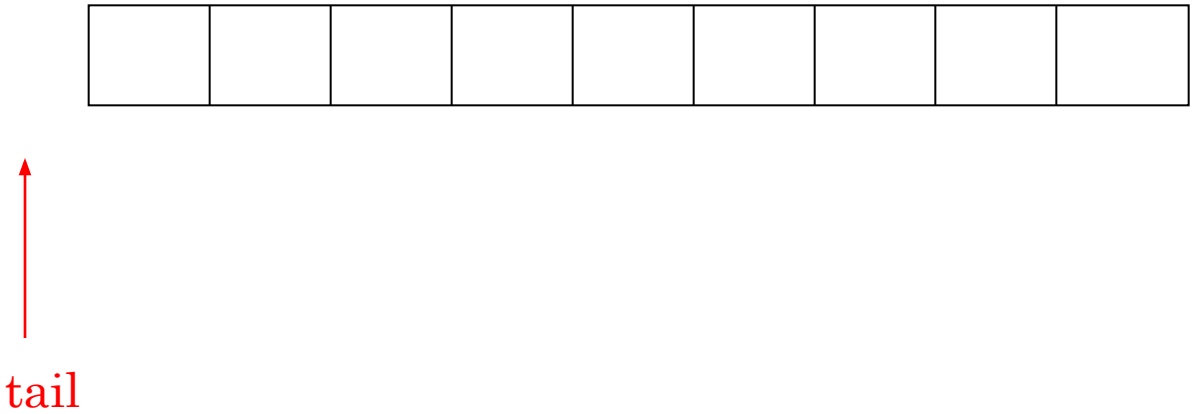
Fila (Simulacao)

Fila (Queue)

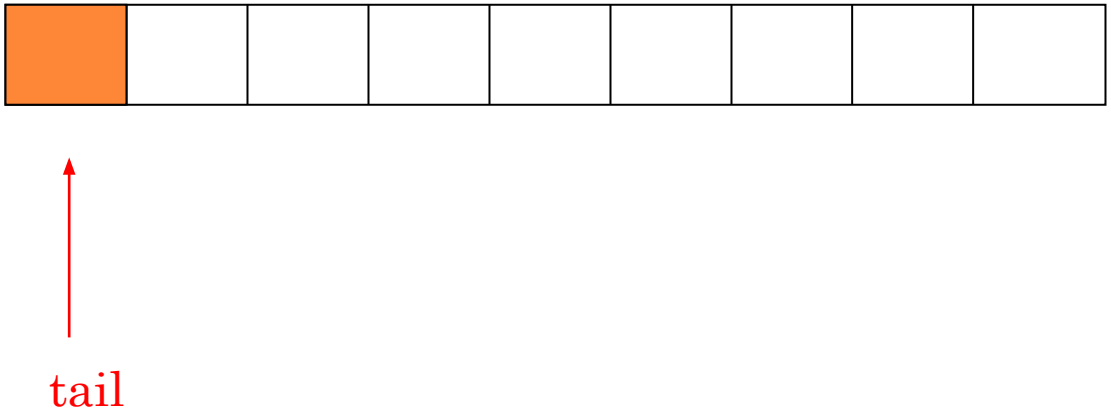
Operações básicas:

- inserir (enqueue)
- remover (dequeue)
- head (peek)
- ~~pesquisar~~
- ~~contem~~
- tamanho (size)
- estaVazio (isEmpty)
- estaCheio (isFull)

Fila (Queue)

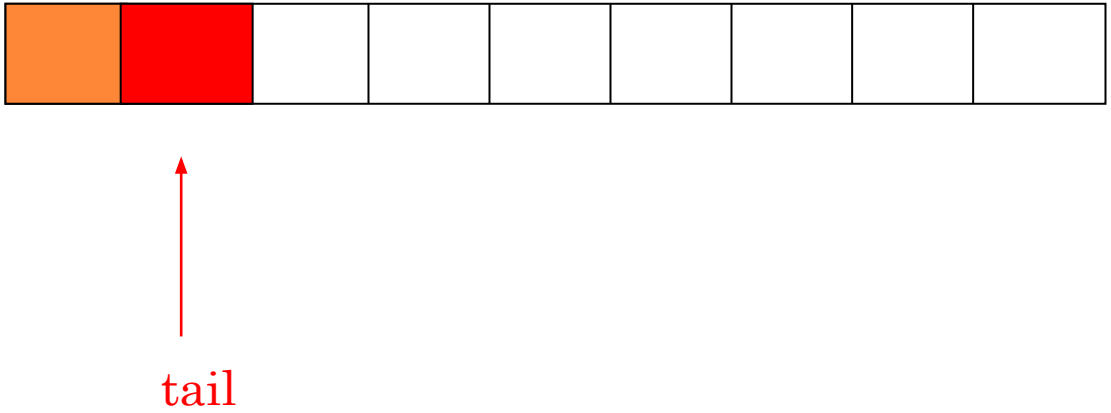


Fila (Queue)



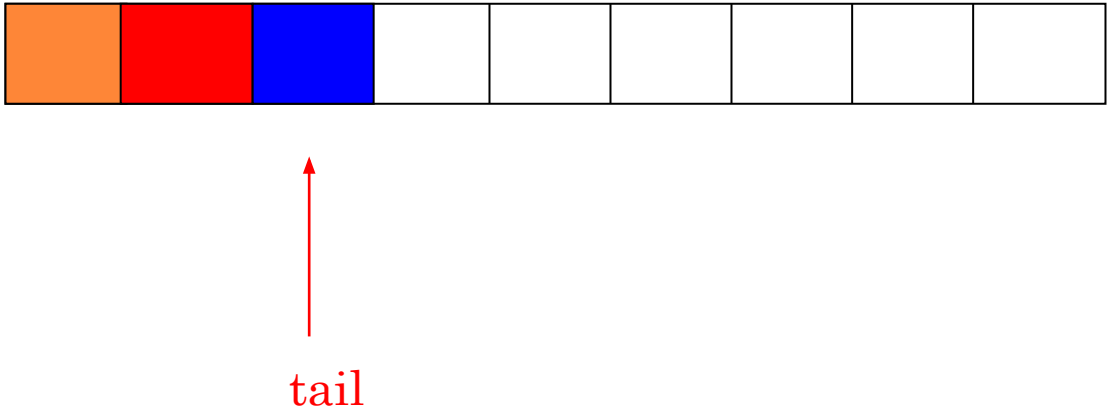
equeue

Fila (Queue)



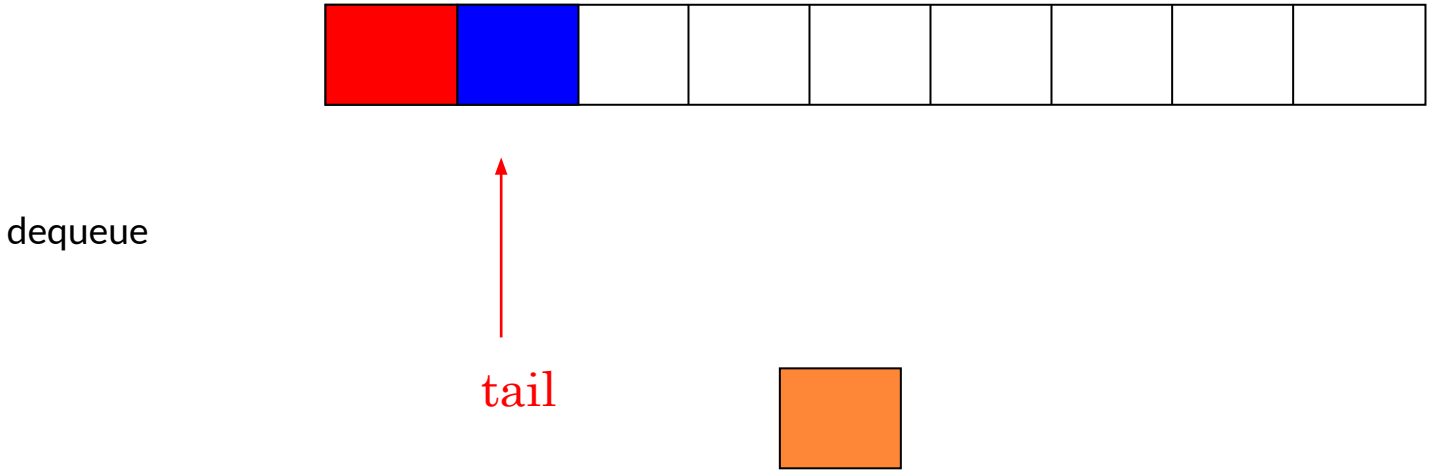
equeue

Fila (Queue)

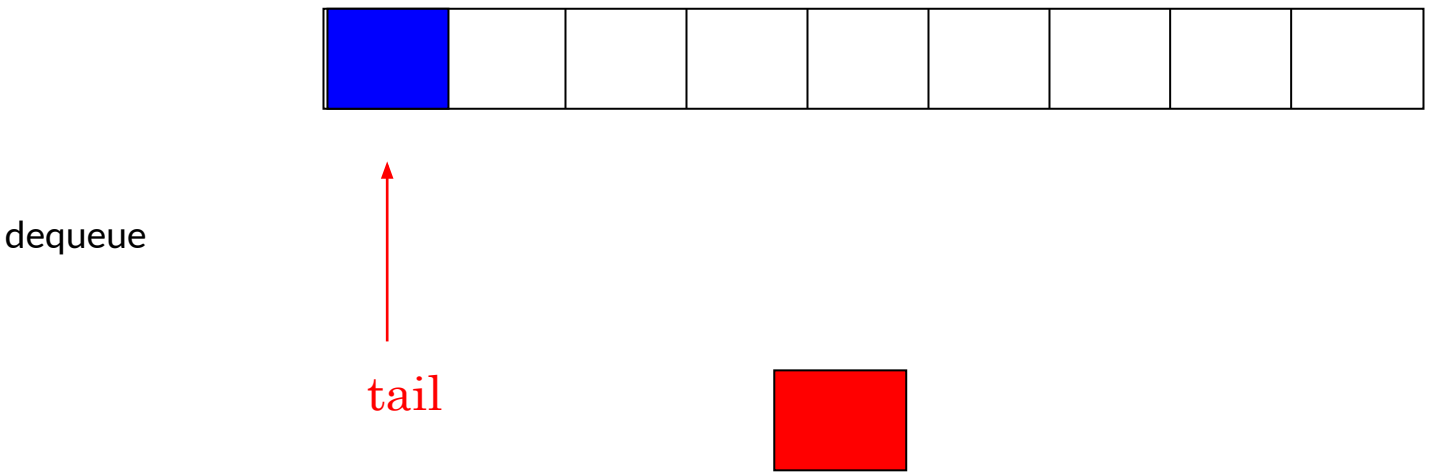


equeue

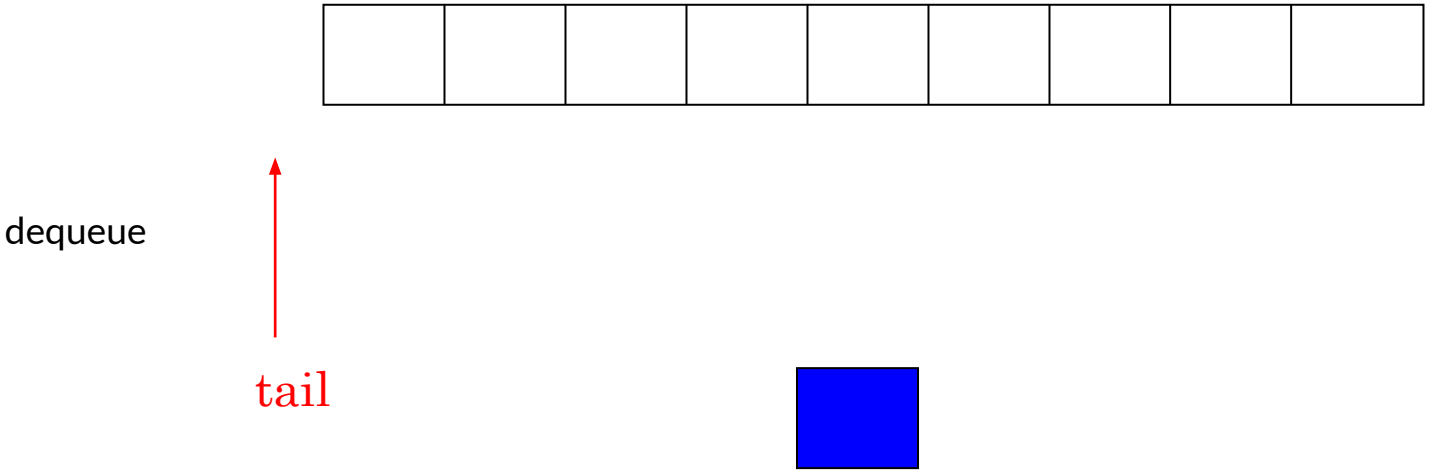
Fila (Queue)



Fila (Queue)



Fila (Queue)



Fila (Queue) - Implementação

```
public interface Queue<T> {
  public void enqueue(T elem) throws QueueOverflowException;
  public T dequeue() throws QueueUnderflowException;
  public T head();
  public boolean isEmpty();
  public boolean isFull();
}
```

```
public class QueueOverflowException
   extends Exception {
 public StackOverflowException() {
  super("Stack is full");
 }
}
```

```
public class QueueUnderflowException
   extends Exception {
 public StackUnderflowException() {
  super("Stack is empty");
 }
}
```

Fila (Queue) - Implementação

```
public class QueueImpl<T> implements Queue<T> {
  private T[] array;
  private int tail;

  public QueueImpl(int size) {
    array = (T[]) new Object[size];
    tail = -1;
  }
  public void enqueue(T elem) throws QueueOverflowException {...}
  public T dequeue() throws QueueUnderflowException {...}
  public T head() {...}
  public boolean isEmpty() {...}
  public boolean isFull() {...}
}
```

Fila (Queue) - Implementação

```
isEmpty(){
   return tail == -1
}
```

```
isFull(){
   return tail == A.length-1
}
```

```
enqueue(item){
   if (not isFull()){
      A[++tail]=item
   }else{
     //error:queue is full
   }
}
```

```
dequeue(){
   if (not isEmpty()){
      result = A[0]
      shiftLeft()
      tail--;
   }else{
     //error:queue is empty
   }
   return result;
}
```

Fila (Queue) - Implementação

```
isEmpty(){
   return tail == -1
}
```

```
isFull(){
   return tail == A.length-1
}
```

```
enqueue(item){
   if (not isFull()){
      A[++tail]=item
   }else{
     //error:queue is full
   }
}
```

```
dequeue(){
   if (not isEmpty()){
      result = A[0]
      shiftLeft()
      tail--;
   }else{
     //error:queue is empty
   }
   return result;
}
```

```
shiftLeft(){
   for i : 0 .. tail{
      A[i] = A[i+1]
   }
}
```

Fila (Queue) - Implementação

```
isEmpty(){
   return tail == -1
}
```

$\theta(1)$

```
isFull(){
   return tail == A.length-1
}
```

$\theta(1)$

```
enqueue(item){
   if (not isFull()){
      A[++tail]=item
   }else{
     //error:queue is full
   }
}
```

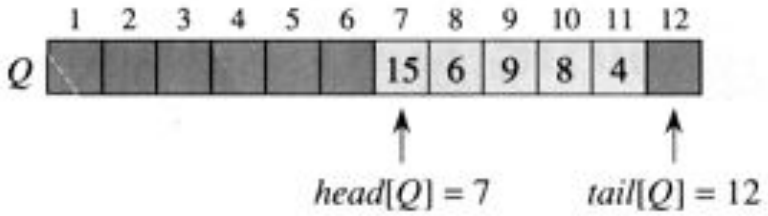
$\theta(1)$

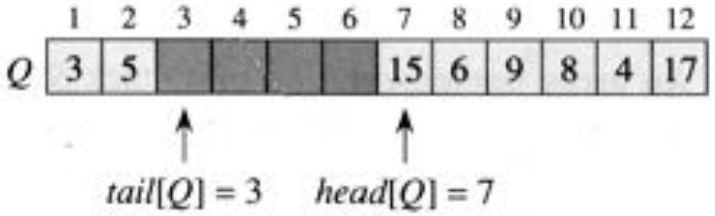
```
dequeue(){
   if (not isEmpty()){
      result = A[0]
      shiftLeft()
      tail--;
   }else{
     //error:queue is empty
   }
   return result;
}
```

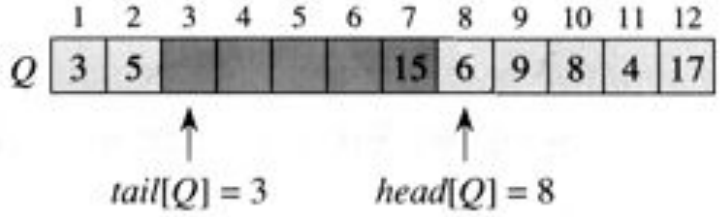
```
shiftLeft(){
   for i : 0 .. tail{
      A[i] = A[i+1]
   }
}
```

$\theta(n)$

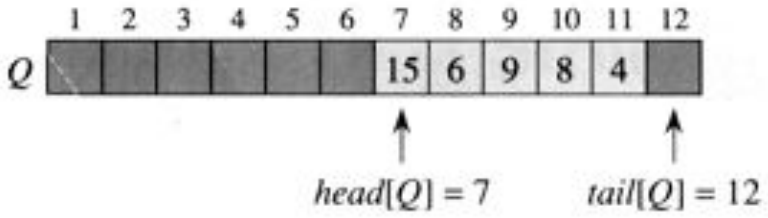
Fila (Queue) - Como evitar o shift?







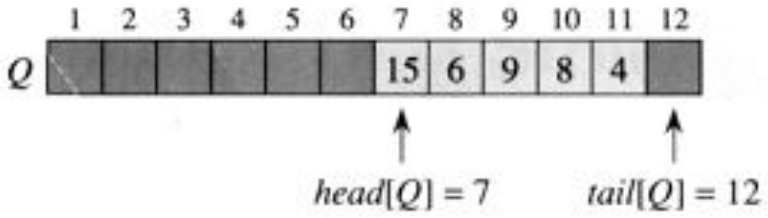
Fila (Queue) - Como evitar o shift?




```
ENQUEUE(Q, x)
Q[tail[Q]] ← x
if tail[Q] = length[Q]
    then tail[Q] ← 1
    else tail[Q] ← tail[Q] + 1
```

```
DEQUEUE(Q)
x ← Q[head[Q]]
if head[Q] = length[Q]
    then head[Q] ← 1
    else head[Q] ← head[Q] + 1
return x
```

Fila (Queue) - Como evitar o shift?



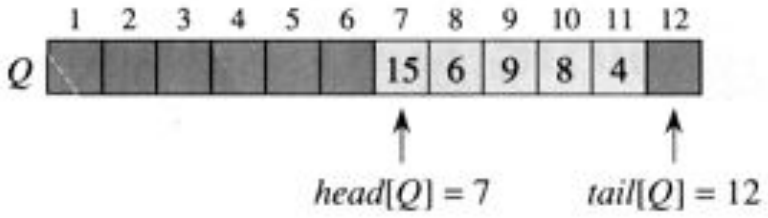
ENQUEUE(Q, x) $\theta(1)$

```
Q[tail[Q]] ← x
if tail[Q] = length[Q]
    then tail[Q] ← 1
    else tail[Q] ← tail[Q] + 1
```

DEQUEUE(Q) $\theta(1)$

```
x ← Q[head[Q]]
if head[Q] = length[Q]
    then head[Q] ← 1
    else head[Q] ← head[Q] + 1
return x
```

Fila (Queue) - Como evitar o shift?




```
ENQUEUE(Q, x)
Q[tail[Q]] ← x
if tail[Q] = length[Q]
    then tail[Q] ← 1
    else tail[Q] ← tail[Q] + 1
```

```
DEQUEUE(Q)
x ← Q[head[Q]]
if head[Q] = length[Q]
    then head[Q] ← 1
    else head[Q] ← head[Q] + 1
return x
```

head = (head + 1) % N

Pilha e Fila

Aplicações e exemplos:

- Filas temporais
- É possível implementar uma Pilha usando duas filas?
- É possível implementar uma Fila usando duas pilhas?
- Com o implementar uma Pilha e Fila ao mesmo tempo usando um único array como estrutura sobrejacente?
- Como implementar uma programa que usa uma Pilha e diz se uma sequência de parênteses é bem formada. Ex:
    - ()(-> false
    - ()()(()) -> true